

Lecture 7: Reinforcement Learning

Satchit Chatterji

SIAS Group, Ivi, Universiteit van Amsterdam

(based on earlier slidesets by Jelle Zuidema & Iris Proff)

Machine Learning and Language Models, 2026



What is Reinforcement Learning?

Markov Decision Processes

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Types of Learning Algorithms

Supervised learning

Signal: labeled data (x, y)

Learn: a predictive mapping

$$\hat{y} = f(x) \approx y$$

Examples: image recognition, sentiment analysis, spam detection...

Unsupervised learning

Signal: unlabeled data x

Learn: structure or representation

$$z = g(x)$$

Examples: document clustering, dimensionality reduction, anomaly detection...

Reinforcement learning

Signal: rewards from interaction

Learn: values or a policy

$Q(s, a) \approx$ future reward,
 $\pi(a | s)$ chooses actions.

Examples: robot navigation, game playing, dialogue systems...

Reinforcement Learning

A structural model from behavioral psychology...

Pavlov (1897) Classical conditioning

Learn a stimulus–outcome contingency

Thorndike (1905) Law of effect / instrumental conditioning

Learn an action–outcome contingency

Tolman (1930) Latent / goal-directed learning

Learn a cognitive map of the environment

Central idea:

Experience changes future behavior according to its consequences.

... formalized by AI as computational models

Behavioral description

Cognitive / AI model

Classical conditioning

→ Rescorla–Wagner model (1972)

Instrumental conditioning

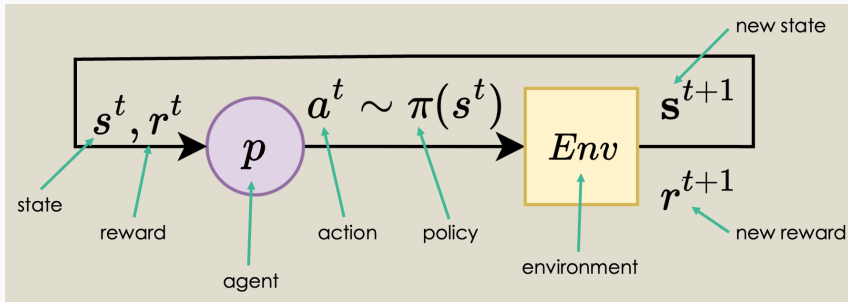
→ Temporal-difference learning (Samuel, 1959)

Q-learning (Watkins & Dayan, 1992)

Goal-directed learning

→ Model-based learning

Agent-Environment Interaction Loop



1. The agent p observes the current state s_t of the environment.
2. It chooses an action a_t to perform in that state, according to its policy $\pi(a \mid s)$.
3. The environment responds with a new state s_{t+1} and a reward r_{t+1} .
4. The agent updates its knowledge based on the new state and reward.

How Do We Define an RL Problem? *Markov Decision Processes*

Definition

A Markov decision process (MDP) is a mathematical model of sequential decision-making:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma).$$

$\mathcal{S}$ states

$\mathcal{A}$ actions

$T(s, a)$ transition function: returns next state after taking a in s

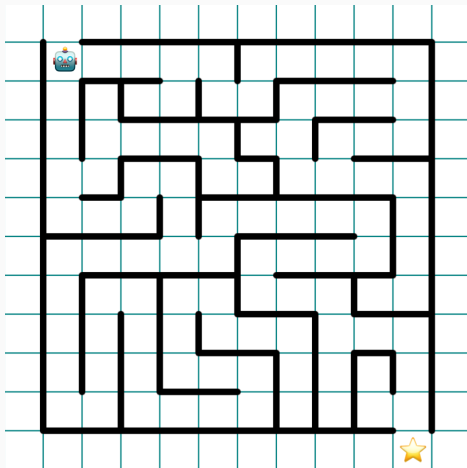
$R(s)$ immediate reward at state s

γ 'discount factor' for future rewards

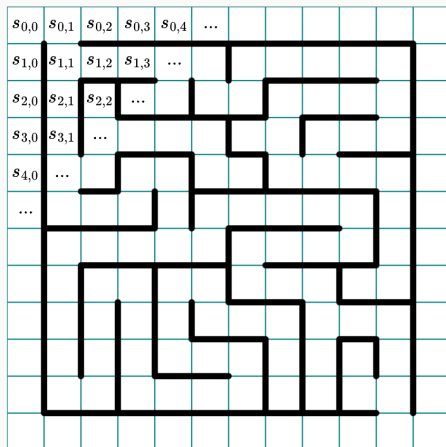
Assumption

Markov property: the next state s' depends only on the current state s and action a .

MDP Example: A Robot in a Maze



MDP Example: State Space



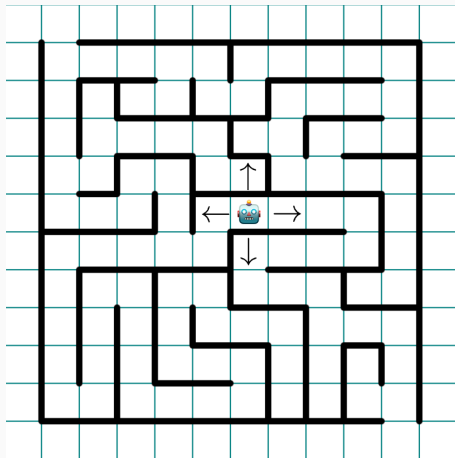
$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma)$$

States: The robot's state is its position in the maze:

$$\mathcal{S} = \{s_{\text{row},\text{col}} \mid \text{row}, \text{col} \in \{0, 1, \dots, 11\}\}.$$

Each state identifies one maze cell.

MDP Example: Action Space



$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma)$$

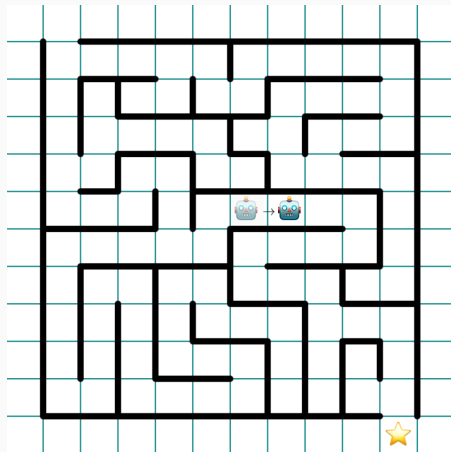
$$\mathcal{S} = \{s_{\text{row}, \text{col}} \mid \text{row}, \text{col} \in \{0, 1, \dots, 11\}\}$$

Actions: At each step, the robot chooses one of four actions:

$$\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}.$$

The action attempts to move the robot to a neighboring cell.

MDP Example: Transition Function



$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma)$$

$$\mathcal{S} = \{s_{\text{row}, \text{col}} \mid \text{row}, \text{col} \in \{0, 1, \dots, 11\}\}$$

$$\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$$

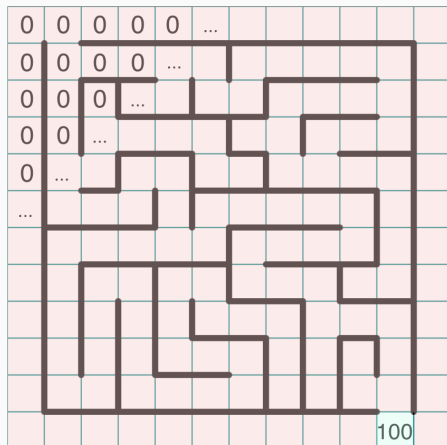
Transition function: The transition function specifies the next state after an action:

$$T(s_{5,6}, \rightarrow) = s_{5,7} \quad (\text{path})$$

$$T(s_{5,6}, \uparrow) = s_{5,6} \quad (\text{wall})$$

$$s_{11,10} = \text{goal} \quad (\text{terminal state})$$

MDP Example: State-Reward Function



$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma)$$

$$\mathcal{S} = \{s_{\text{row}, \text{col}} \mid \text{row}, \text{col} \in \{0, 1, \dots, 11\}\}$$

$$\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$$

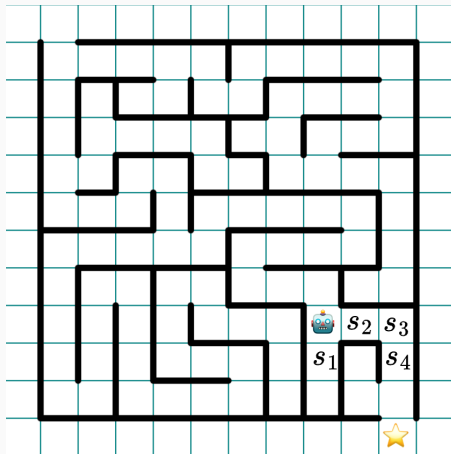
$T(s, a)$: how actions move the robot

Reward function: The reward depends on the state reached:

$$R(s) = \begin{cases} 100, & s = s_{11,10}, \\ 0, & \text{otherwise.} \end{cases}$$

The goal state is valuable; all other states have zero immediate reward.

MDP Example: γ -Discounting and Utility



$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma)$$

$$\mathcal{S} = \{s_{\text{row}, \text{col}} \mid \text{row}, \text{col} \in \{0, 1, \dots, 11\}\}$$

$$\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$$

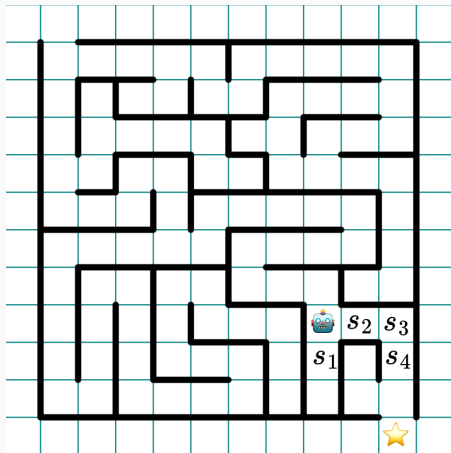
$T(s, a)$: how actions move the robot

$R(s)$: 100 for ★, 0 otherwise

Question 1

Consider states s_1 to s_4 . Which states are reachable from the current state?

MDP Example: γ -Discounting and Utility



$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma)$$

$$\mathcal{S} = \{s_{\text{row}, \text{col}} \mid \text{row}, \text{col} \in \{0, 1, \dots, 11\}\}$$

$$\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$$

$T(s, a)$: how actions move the robot

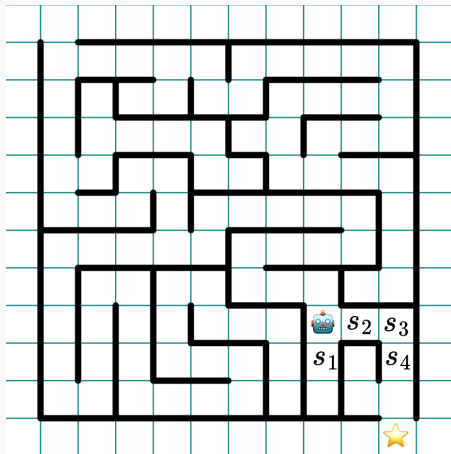
$R(s)$: 100 for ★, 0 otherwise

Question 2

What are the immediate rewards for s_1 and s_2 ?

How does the 'value' (utility) of s_1 compare to the value of s_2 ?

MDP Example: γ -Discounting and Utility



$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma)$$

$$\mathcal{S} = \{s_{\text{row}, \text{col}} \mid \text{row}, \text{col} \in \{0, 1, \dots, 11\}\}$$

$$\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$$

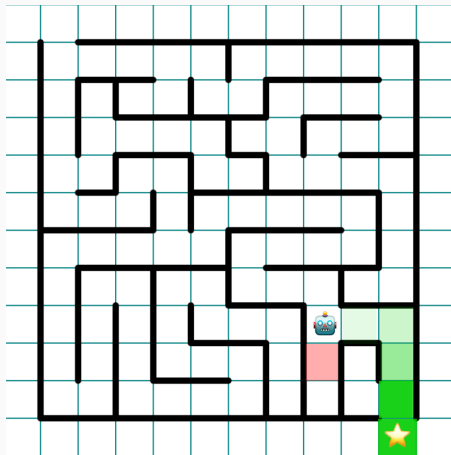
$T(s, a)$: how actions move the robot

$R(s)$: 100 for ★, 0 otherwise

Question 3

How does $U(s_2)$ compare to $U(s_3)$? How does $U(s_3)$ compare to $U(s_4)$?

MDP Example: γ -Discounting and Utility



$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma)$$

$$\mathcal{S} = \{s_{\text{row}, \text{col}} \mid \text{row}, \text{col} \in \{0, 1, \dots, 11\}\}$$

$$\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$$

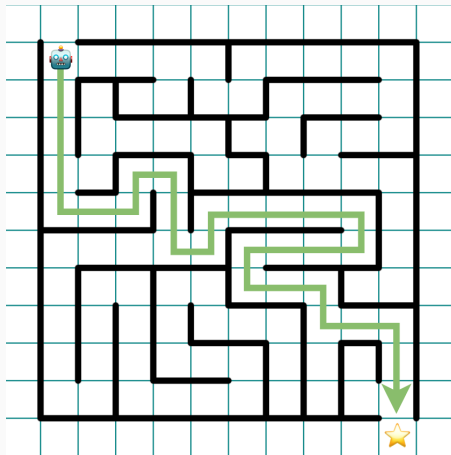
$T(s, a)$: how actions move the robot

$R(s)$: 100 for ★, 0 otherwise

Discount factor: Future rewards are worth less than immediate rewards:

$$\gamma \in [0, 1)$$

MDP Example: Bellman Optimality



$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma)$$

$$\mathcal{S} = \{s_{\text{row}, \text{col}} \mid \text{row}, \text{col} \in \{0, 1, \dots, 11\}\}$$

$$\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$$

$T(s, a)$: how actions move the robot

$R(s)$: 100 for ★, 0 otherwise

γ : discount factor for future rewards, $0 \leq \gamma < 1$

Utility (Bellman): immediate reward plus discounted future utility:

$$U^*(s) = R(s) + \gamma \max_{a \in \mathcal{A}} U^*(T(s, a))$$

Goal: learn a policy $\pi^*(s)$ that maximizes expected discounted future reward:

$$\pi^*(s) = \arg \max_{a \in \mathcal{A}} U^*(T(s, a))$$

What is Reinforcement Learning?

Markov Decision Processes

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

From State Values to Action Values

State value: How good is it to be in state s ?

$$U^*(s) = \max_{a \in \mathcal{A}} Q^*(s, a)$$

Action value: How good is it to take action a in state s ?

$$Q^*(s, a) = R(s) + \gamma U^*(T(s, a))$$

Why learn Q ?

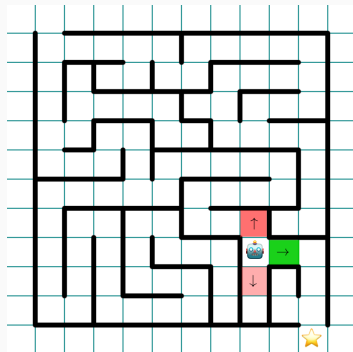
$Q(s, a)$ lets the agent compare actions directly and choose

$$\pi^*(s) = \arg \max_a Q^*(s, a).$$

Q-learning learns these action values directly from experience, without first knowing the transition function T .

Recall: Utility à la Bellman

$$U^*(s) = R(s) + \gamma \max_{a \in \mathcal{A}} U^*(T(s, a))$$



Q-Learning Update Rule (*Temporal-Difference Learning*)

The Q-learning update rule is a form of *temporal-difference learning*, which updates the action value $Q(s, a)$ based on the observed reward and the estimated future rewards. (Watkins & Dayan, 1992)

Q-value update

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \overset{\text{learning rate}}{\downarrow \alpha} \delta_t$$

Prediction (TD) error

$$\delta_t = \overbrace{R(s_{t+1}) + \gamma \max_{a \in \mathcal{A}} Q_t(s_{t+1}, a)}^{\text{target}} - \overbrace{Q_t(s_t, a_t)}^{\text{current estimate}}$$

Convergence in the limit

$$\lim_{t \rightarrow \infty} Q_t(s, a) = Q^*(s, a)$$

For a deterministic transition function: $Q^*(s, a) = R(s) + \gamma \max_{a' \in \mathcal{A}} Q^*(T(s, a), a')$.

Q-Learning Action-Selection Policy

Suppose we are in state s , with possible actions a_1 and a_2 , and

$$Q(s, a_1) = 0.5, \quad Q(s, a_2) = 0.7.$$

Which action should we choose?

Q-Learning Action-Selection Policy

Suppose we are in state s , with possible actions a_1 and a_2 , and

$$Q(s, a_1) = 0.5, \quad Q(s, a_2) = 0.7.$$

Which action should we choose?

0.5		0.7		
-----	---	-----	--	---

Q-Learning Action-Selection Policy

Suppose we are in state s , with possible actions a_1 and a_2 , and

$$Q(s, a_1) = 0.5, \quad Q(s, a_2) = 0.7.$$

Which action should we choose?

0.7		0.5		
-----	---	-----	--	---

Suppose we are in state s , with possible actions a_1 and a_2 , and

$$Q(s, a_1) = 0.5, \quad Q(s, a_2) = 0.7.$$

Which action should we choose?

Question 4

When should an agent exploit the action it currently believes is best, and when should it explore a less certain action that might lead to greater reward?

Exploration–Exploitation Trade-Off



Exploit what you know

or



Explore new options

An agent has to exploit what it has experienced in order to obtain reward, but it also has to explore in order to make better action selections in the future.

Sutton & Barto (2018) Classic example: [multi-armed bandit](#)

Multi-Armed Bandit



At each round, a gambler chooses one of several slot machines. Each machine has an unknown reward distribution; the goal is to maximize total reward while learning which machine is best.

Q-Learning Exploration Policies

Suppose $Q(s, a_1) = 0.5$ and $Q(s, a_2) = 0.7$. How should we trade off exploration and exploitation?

Epsilon-greedy

$$\mathbb{P}(a_i | s) = \begin{cases} 1 - \epsilon & , \text{ if } a_i = \arg \max_{a' \in \mathcal{A}} Q(s, a'), \\ \frac{\epsilon}{|\mathcal{A}| - 1} & , \text{ otherwise.} \end{cases}$$

With probability $1 - \epsilon$ choose the greedy action; with probability ϵ choose uniformly from the remaining. For $\epsilon \rightarrow 0$ the policy is greedy; for $\epsilon \rightarrow 1$ it is uniform.

Softmax/Boltzmann

$$\mathbb{P}(a_i | s) = \frac{e^{Q(s, a_i)/\tau}}{\sum_{a' \in \mathcal{A}} e^{Q(s, a')/\tau}}$$

Exponentially scale the action values and normalize to get a probability distribution. For $\tau \rightarrow 0$ the policy is greedy; for $\tau \rightarrow \infty$ it is uniform.

Q-Learning Pseudo-Code

1. Set $\alpha \in (0, 1]$, $\gamma \in [0, 1)$, and small $\epsilon \in (0, 1]$ or $\tau > 0$.
2. Initialize $Q(s, a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$.
3. Initialize s to a starting state.
4. For each episode:
 - 4.1 While s is not terminal:
 - 4.1.1 Choose a from s using an exploration policy.
 - 4.1.2 Observe reward R and next state s' .
 - 4.1.3 Update Q and set $s \leftarrow s'$.
 - 4.1.4 Update α and ϵ or τ if desired.

What is Reinforcement Learning?

Markov Decision Processes

Q-learning

RL in the brain

Model-based RL

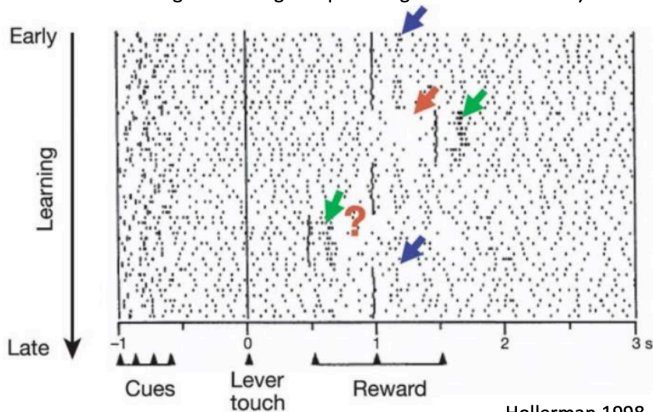
Extensions of Q-learning

Deep RL

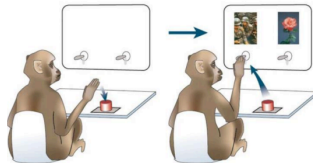


Temporal-Difference Learning in the Brain

Recording from a single dopaminergic neuron in monkey brain



Hollerman 1998



Upper blue: expected reward
Red: expected reward omitted
Green: unexpected early/late reward
Lower blue: expected reward

What is Reinforcement Learning?

Markov Decision Processes

Q-learning

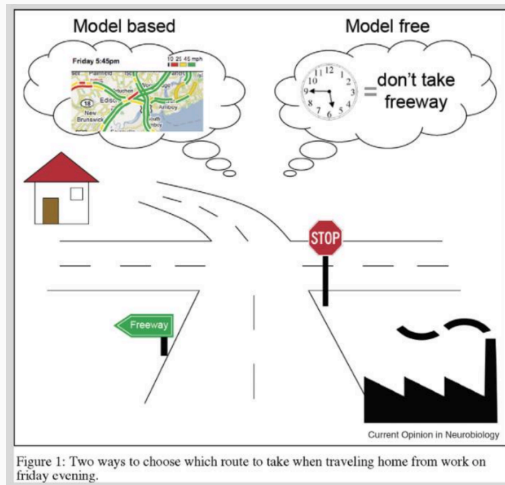
RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Model-Based Versus Model-Free Learning



Model-Based Reinforcement Learning

Environment model

$$M: (s, a) \mapsto (s', r)$$

The agent learns or uses a model that predicts the next state and reward after an action.

Planning

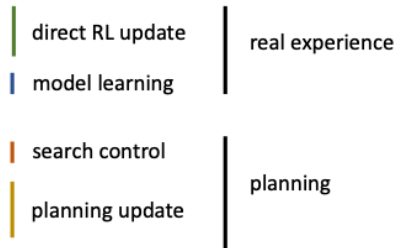
It simulates possible action sequences with this model, then chooses the action with the best predicted outcome.

Trade-off

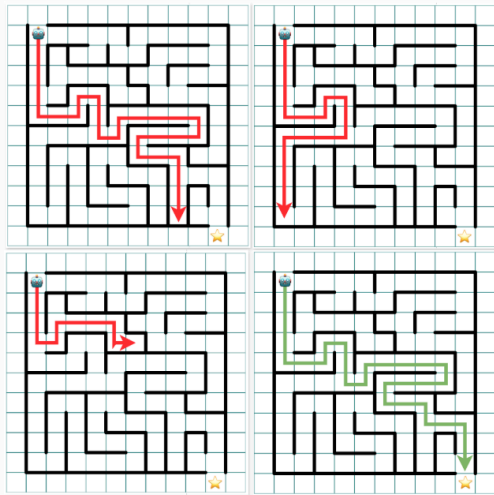
Model-based RL can adapt quickly when conditions change, but planning is computationally expensive and depends on an accurate model.

Dyna-Q Pseudo-Code

```
initialize parameters
for number of episodes
  while s is not a terminal state
    choose a from s using action selection policy
    observe R and s', update Q, update s
    update model  $M(s,a) = (R,s')$ 
    repeat n times
      choose (s,a) to simulate
      simulate action: observe R and s'
      update Q
```



Model-Based Learning: Dyna-Q



What is Reinforcement Learning?

Markov Decision Processes

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Extensions of Base RL Algorithms

- **Decaying learning rate:** α decreases over time.
- **Balancing exploration:** ϵ or τ decreases over time.
- **n -step returns:** use several future rewards before bootstrapping.
- **Eligibility traces:** update not only the last state–action pair, but all previously visited pairs with a decay factor λ .
- **Experience replay:** store transitions and learn from them more than once.
- **Function approximation:** represent $Q(s, a)$ with a parameterized function $Q_\theta(s, a)$.
- **Double Q-learning:** reduce overestimation from the maximization step.
- **Hybrid models:** combine model-based and model-free components.
- **Policy gradients:** learn a policy $\pi_\theta(a \mid s)$ directly, rather than learning $Q(s, a)$.
- **Deep RL:** use deep neural networks to represent $Q(s, a)$ or $\pi(a \mid s)$ to handle high-dimensional state spaces.

What is Reinforcement Learning?

Markov Decision Processes

Q-learning

RL in the brain

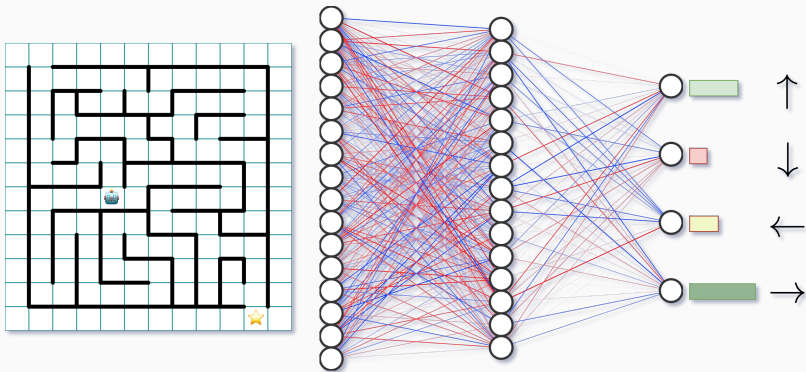
Model-based RL

Extensions of Q-learning

Deep RL

Deep Reinforcement Learning (DQN, Mnih et al. 2015)

- **Architecture:** Replace Q-table with a neural network Q_θ , with weights θ ; use it to estimate $Q(s, a)$. Generally, the input is a state s and the output is a vector of Q-values, one for each action a .



Deep Reinforcement Learning (DQN, Mnih et al. 2015)

- **Training:** Remember that the Q-learning update rule is:

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha \left(R(s_{t+1}) + \gamma \max_{a \in \mathcal{A}} Q_t(s_{t+1}, a) - Q_t(s_t, a_t) \right)$$

with the prediction error being the difference between the target and the current estimate:

$$\delta_t = \overbrace{R(s_{t+1}) + \gamma \max_{a \in \mathcal{A}} Q_t(s_{t+1}, a)}^{\text{target}} - \overbrace{Q_t(s_t, a_t)}^{\text{current estimate}}$$

- In DQN, we replace the Q-table with a neural network $Q_\theta(s, a)$ and minimize the prediction error using a separate target network $Q_{\bar{\theta}}$:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim D} \left[\left(R(s_{t+1}) + \gamma \max_{a \in \mathcal{A}} Q_{\bar{\theta}}(s_{t+1}, a) - Q_\theta(s_t, a_t) \right)^2 \right]$$

Deep Reinforcement Learning (DQN, Mnih et al. 2015)

- **Experience replay:** act in the world (choose actions using ϵ -greedy) and gather data. Save transitions (s, a, r, s') in a replay buffer. Then uniformly sample mini-batches of transitions from the buffer to train the network. This breaks correlations between sequential data and stabilizes training.
- Training is done by stochastic gradient descent on the loss function $\mathcal{L}(\theta)$, using the sampled mini-batches:

$$\nabla \mathcal{L}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla \left(R(s_{t+1}) + \gamma \max_{a \in \mathcal{A}} Q_{\bar{\theta}}(s_{t+1}, a) - Q_{\theta}(s_t, a_t) \right)^2$$

DQN Stabilization: Target Networks

Two networks

The **online network** Q_θ learns from gradient updates. The **target network** $Q_{\bar{\theta}}$ provides a temporarily fixed target.

$$y_t = R(s_{t+1}) + \gamma(1 - d_t) \max_{a \in \mathcal{A}} Q_{\bar{\theta}}(s_{t+1}, a)$$

Slowly update the target

Every K gradient steps, copy the online weights: $\bar{\theta} \leftarrow \theta$. Here $d_t = 1$ when the next state is terminal.

Why?

Keeping the target fixed for several updates makes learning more stable: the network is not chasing a target that changes at every step.

Deep Reinforcement Learning (DQN, Mnih et al. 2015)

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

 With probability ϵ select a random action a_t

 otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

 Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

 Every C steps reset $\hat{Q} = Q$

End For

End For

LETTER

Human-level control through deep reinforcement learning

Vikashroyt Mishra*, Sergey Sidorovskiy*, David Silver*, Andrej A. Rusu*, Joel Veness*, Marc G. Bellemare*, Alex Guez*, Martin Riedmiller*, Florian H. Fiedler*, Georg Ostrovski*, Stig Petersen*, Charles Beattie*, Andre Sallik*, Martin Amundson*, Hsien King*, Orlanion Karamer*, Duan Wianter*, Shane Legg†, David Hasselblad†

The theory of reinforcement learning provides a normative account, deeply rooted in psychological and neuroscience, perspective on animal behaviour, of how agents may optimise their control of an environment. To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must derive efficient representations of the environment from high-dimensional sensory inputs, and use these to generalise past experience to new situations. Remarkably, humans and other animals seem to solve this problem through a harmonious combination of reinforcement learning and hierarchical sensory processing systems¹, the former evidenced by a wealth of neural data revealing notable parallels between the plastic signals emitted by dopamine neurons and temporal difference reinforcement learning algorithms². While reinforcement learning agents have achieved some successes in a variety of domains³, their applicability has previously been limited to domains in which useful features can be hand-engineered or domains with fully observed, low-dimensional state spaces. Here we use recent advances in training deep neural networks^{4,5,6,7,8,9} to develop a novel artificial agent, termed a deep Q-network, that can learn successful policies directly from high-dimensional sensory input using only its end reinforcement learning. We tested this agent on the challenging domain of classic Atari 2600 games¹⁰. We demonstrate that the deep Q-network agent, receiving only the pixels and the game score as inputs, was able to surpass the performance of all previous algorithms and achieve a level comparable to that of a professional human games player across a set of 49 games, using the same algorithm, network architecture and hyperparameters. This result bridges the divide between high-dimensional sensory inputs and actions, resulting in the first artificial agent that is capable of learning to excel at a diverse array of challenging tasks.

We set out to create a single algorithm that would be able to develop a wide range of successful policies on a varied domain of challenging tasks – our goal of general artificial intelligence¹¹ that has eluded previous researchers¹². To achieve this, we developed an agent, a deep Q-network (DQN), which is able to combine reinforcement learning with a class of artificial neural networks¹³ known as deep neural networks. Notably, recent advances in deep neural networks^{14,15,16,17,18,19,20,21,22} in which several layers of nodes are used to build up progressively more abstract representations of the data, have made it possible for artificial neural networks to learn concepts such as object categories directly from raw sensory data. We use one particularly successful architecture, the deep convolutional network²³, which uses hierarchical layers of filter convolutional filters to mimic the effects of receptive fields – inspired by Hubel and Wiesel’s seminal work on hierarchical processing of visual scenes²⁴ – thereby enabling the local spatial correlations present in images, and building up abstractions to natural transformations such as changes of viewpoint or scale.

We consider tasks in which the agent interacts with an environment through a sequence of observations, actions and rewards. The goal of the

agent is to select actions in a fashion that maximises cumulative future reward. More formally, we use a deep convolutional neural network to approximate the optimal action-value function

$$Q^*(s,a) = \max_{a'} E \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_t = s, a_t = a, s_{t+1} = s' \right]$$

which is the maximum over all possible actions of the expected return, i.e. the expected sum of discounted future rewards, of each time step t , achieved by a behaviour policy $\pi = P(a_t|s_t)$ after making an observation (s) and taking an action (a) (see Methods). Reinforcement learning is known to be unstable or even to diverge when a nonlinear function approximator such as a neural network is used to represent the action-value (also known as Q -function)²⁵. This instability has several causes: the correlations present in the sequence of observations, the fact that small updates to Q may significantly change the policy and therefore change the data that are seen, and the correlation between the action values (Q) and the target values $y = \max_{a'} Q(s', a')$. We address these instabilities with a novel variant of Q -learning which uses two key ideas. First, we used a biologically inspired mechanism termed experience replay²⁶ that randomises over the data, thereby removing correlations in the observation sequence and smoothing over changes in the data distribution (see below for details). Second, we used an iterative update that adjusts the action values (Q) towards target values that are only periodically updated, thereby reducing correlations with the target.

While other stable methods exist for training neural networks in the reinforcement learning setting, such as neural fixed Q -iteration²⁷, these methods involve the repeated training of networks, all over on hundreds of iterations. Consequently, these methods, unlike our algorithm, are not sufficient to be used successfully with large neural networks. We parameterise an approximate value function $Q(s,a,k)$ using the deep convolutional neural network shown in Fig. 1, in which the free parameters (that is, weights) of the Q -network at iteration k . To perform experience replay we store the agent’s experiences $\epsilon = (s_{k-2}, a_{k-2}, r_{k-2}, s_{k-1})$ at each time step t in a data set $\mathcal{D}_k = \{ \epsilon_i \}_{i=0}^{k-1}$. During learning, we apply Q -learning updates, on samples (or sub-batches) of experience $\epsilon_i = (s_{i-2}, a_{i-2}, r_{i-2}, s_{i-1})$ chosen uniformly at random from the pool of stored samples. The Q -learning update at iteration k uses the following loss function:

$$L(k) = \frac{1}{|\mathcal{D}_k|} \sum_{\epsilon \in \mathcal{D}_k} \left[\left(r + \gamma \max_{a'} Q(s', a, k) - Q(s, a, k) \right)^2 \right]$$

in which L is the discount loss determining the agent’s losses, a , are the parameters of the Q -network at iterations k and γ are the network parameters used to compute the target at iteration k . The target network parameters Q' are only updated with the Q -network parameters Q every C steps and are held fixed between individual updates (see Methods).

To evaluate our DQN agent, we took advantage of the Atari 2600 games, which offer a diverse array of tasks in 4×4 designed to be

NEWS & VIEWS

TOPIC: INTELLIGENCE

Learning to see and act

An artificial-intelligence system uses machine learning from massive training sets to teach itself to play 49 classic computer games, demonstrating that it can adapt to a variety of tasks. See Letters p.529

BERNARD SCHOLKOPF

Improvements in our ability to process large amounts of data have led to progress in many areas of science, not least artificial intelligence (AI). With advances in machine learning, has come the development of machines that can learn intelligent behaviour directly from data, rather than being explicitly programmed to exhibit such behaviour. For instance, the advent of ‘big data’ has resulted in systems that can recognise objects to sounds with considerable precision. On page 529 of this issue, Mnih et al.¹ describe an agent that uses large data sets to teach itself how to play 49 classic Atari 2600 computer games by looking at the pixels and learning actions that increase the game score. It beat a professional games player in many games² – a remarkable example of the progress being made in AI.

In machine learning, systems are trained to infer patterns from observational data. A particularly simple type of pattern, a mapping between inputs and outputs, can be learnt through a process called supervised learning. A supervised learning system is given training data consisting of example inputs and the corresponding outputs, and comes up with a

model to explain these data (a process called function approximation). It does this by choosing from a class of model specified by the system’s designer. Designing this class is an art: its size and complexity should reflect the amount of training data available, and its content should reflect ‘prior knowledge’ that the designer of the system considers useful for the problem at hand. If all this is done well, the inferred model will have ‘applied not only for training set, but also for other data that adhere to the same underlying pattern.

The rapid growth of data sets means that machine learning can now use complex model classes and tackle highly non-trivial inference problems. Such problems are usually characterised by several factors: the data are multidimensional, the underlying pattern is unknown, it might be nonlinear and/or non-stationary, and the designer has only weak prior knowledge about the problem – in particular, a mechanistic understanding is lacking. The human brain repeatedly solves non-trivial inference problems as we go about our daily lives, interpreting high-dimensional sensory data to determine how to control all the muscles of the body. Single supervised learning is clearly not the whole story, because

we often learn without a ‘supervisor’ telling us the outputs of a hypothetical input-output function. Here, reinforcement³ has a central role in learning behaviours from multiple experiences. Machine learning adopted this idea to develop reinforcement learning algorithms⁴, in which supervision takes the form of a numerical reward signal⁵, and the goal is for the system to learn a policy that gives the current state, determines which action to pick to maximise an accumulated future reward.

Mnih et al. use a form of reinforcement learning known as Q -learning⁶ to teach systems to play a set of 49 vintage video games, learning how to increase the game score as a numerical reward. In Q -learning, Q represents the accumulated future reward, Q' if in state s the system first performs action a , and subsequently follows an optimal policy. The system tries to approximate Q' by using an artificial neural network – a function approximator known as experience replay⁷ by using a neural network – called a deep Q -network (DQN). The DQN input (the pixels from four consecutive game screens) is processed by convolutional hidden layers of computations, which extract more and more specialised visual features to help approximate the complex

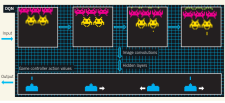


Figure 1 | Computer game. Mnih et al.¹ have designed an artificial-intelligence system, using the Q -network (DQN), that teaches itself to play 49 video games. The DQN analyses a sequence of four game screens (input) and approximates the value of the next action (output). The consequence on the later game screen is that action is taken and followed by the best possible course of subsequent actions. The first layers of the DQN analyse the pixels of the game screen and extract information from more and more specialised neural features (image convolutions). Subsequent, fully-connected hidden layers predict the value of the next action. These features, the action and approximation, are then pooled together to make the final decision on the specific game the system is playing, representing this in the same way as each of the 49 games.

*Google DeepMind, 1 New Street Square, London EC4A 3DF, UK. †These authors contributed equally to this work.

Deep Reinforcement Learning (DQN, Mnih et al. 2015)

A tutorial on deep reinforcement learning

<https://wiki.pathmind.com/deep-reinforcement-learning>

Reinforcement Learning for Language Models

An RL problem over generated text

- **State:** a prompt and the tokens generated so far.
- **Action:** the next token, sampled from $\pi_{\theta}(\text{token} \mid \text{context})$.
- **Reward:** a human preference, AI feedback, a verifier, or task success after a completion.

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} [r(x, y)]$$

Examples

RLHF and RLAIIF use preference feedback; policy-gradient methods such as PPO or GRPO update the language-model policy.

- **World models:** learn predictive simulators and plan through imagined futures.
- **Offline RL:** learn from logged data when online trial and error is costly or unsafe.
- **Foundation-model agents:** combine pretrained language and vision models with RL for planning and tool use.
- **Safe RL:** maximize reward while respecting constraints during learning and deployment.
- **Multi-agent RL:** learn to coordinate, compete, and communicate with other agents.

Central challenge:

Build agents that generalize reliably beyond their training environments and improve safely from limited real-world interaction.